BECKETT

DESIGN PROPOSAL · OPS-151 · --project beckett

Run of Show
a workflow engine the concierge conducts

A proposal to retire Plane as Beckett's execution backend and replace its one fixed ticket lifecycle with a per-task, concierge-authored workflow: stages, iteration budgets, parallel fan-out and gates composed at filing time — not compiled into the dispatcher.

Author
Beckett — worker seat
Claude Fable 5 · high

Requested by
ro (owner)
Discord · OPS-151

Date
12 July 2026
rev 1

Status
Proposal — no code
grounded in v3.2 source

THE PROPOSAL · WORKFLOW AS DATA

1

The proposal — a per-task flow spec

Replace the compiled-in lifecycle with a small, validated document — the **flow spec**, or *run of show* — attached to every task at filing time. The dispatcher's hardcoded state→action table becomes a generic interpreter over exactly **four node kinds**. Everything below the node boundary (worktrees, casting, envelopes, done-signals, journal, steering, courier) is reused unchanged.

3.1 Design principles

- **Small algebra, closed set.** Four node kinds: `worker`, `gate`, `fanout`, `join`. No expressions, no conditionals beyond pass/fail edges, no user-defined node types. Anything fancier must arrive as a fifth kind with its own design doc — that is the fence against the workflow-engine tarpit (§7.1).
- **Every loop is bounded by construction.** A spec that could run forever does not validate. Caps are per-node fields, defaulted, not global constants.
- **Today's behaviour is two specs.** The OPS and INT lifecycles must be expressible byte-for-byte (Appendix A), so cutover is a no-op until the concierge starts writing new shapes.
- **The concierge composes; the engine conducts.** Authoring is data + presets (§5); execution, recovery and budget enforcement are the engine's (§4). The worker never sees the flow — it still gets a brief, a worktree, and a done-signal schema.

3.2 The data model

The spec travels in a fenced `beckett-flow` block exactly where `beckett-cast` lives today (and, after migration, as a typed field in the local task record — §6). Casting stays a separate concern: the flow names *seats*; the cast fills them. A flow with no `beckett-flow` block is compiled from the cast exactly as today (effort → gate), so existing filings never break.

3.3 Validation — what the linter refuses at filing time

- Unknown edge targets; unreachable nodes; an `entry` that isn't a node; fanout arms whose edges escape their join.
- **Unbounded cycles:** every cycle in the graph must pass through a node whose `maxVisits` / `maxFails` is finite (all default finite; setting one to `Infinity` is simply not representable).
- Budget sanity: `maxConcurrent` ≤ global `max_workers`; a Fable seat anywhere in the spec triggers the existing confirm-before-cast handshake — flow authoring does not bypass the cast doctrine.
- Cast validation per seat via the existing `validateCasting` (blocked models, malformed efforts) `presets.ts:76–112`.

3.4 What today's shapes look like (proof of containment)

The default reviewed lifecycle, as a spec — this is *all* of it:

One-pass work is the same spec minus the gate (`onPass:"done"`). The INT design flow adds two nodes in front — a `design` worker and a `human` gate — instead of two enum values, a board, and five modules. All three appear in full in Appendix A.

3.5 And what today cannot do at all

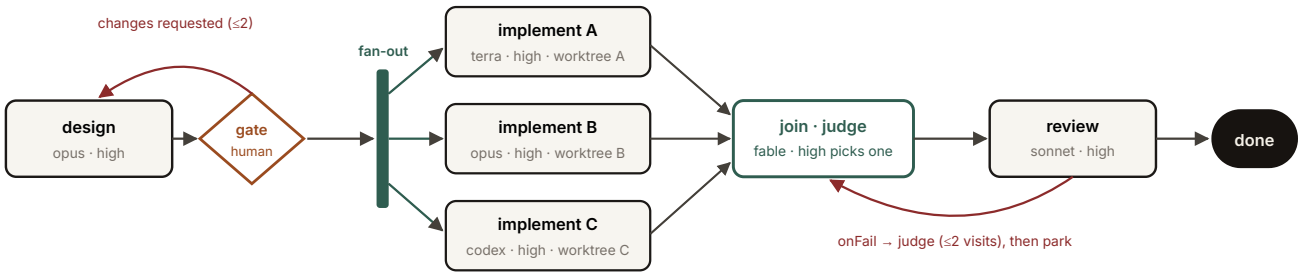

Fig. 3 — a bake-off run of show. Design → human gate → three isolated candidate implementations → a judge join that picks (or synthesises) a winner → fresh review → done. Six nodes of data. Under the current engine this is three hand-filed tickets, a human doing the judging, and no joint lineage.